PROJECT SYNOPSIS

on

“SMART TICKET UNDERSTANDING ENGINE”

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

S No	Reg No	Name of the Student
1	237Y1A0540	NAYAKAM SATYAJEET
2	237Y1A0570	JAKKU HARIKA
3	237Y1A0574	SHERLA LOHITH KUMAR
4	237Y1A05A5	MAMIDALA VARUN

Under the Guidance of

Dr. PARVEZ

Associate Professor, Department of **CSE**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Supervisor **Project Coordinator** **HOD**

Marri Laxman Reddy Institute of Technology and Management

September, 2026

Project Batch No: MP-A05

1. Title of the Project

- Smart Ticket Understanding Engine: An AI-Powered NLP System for Intent Classification, Sentiment Analysis, SLA-Based Prioritization and Automated Routing of Customer Support Tickets

Project Guide: Dr. Parvez, Associate Professor, Department of CSE

2. Abstract

Customer support centers receive large volumes of unstructured tickets through emails, chats and web forms, and manual triage of these tickets is slow, inconsistent and difficult to scale. This often leads to Service Level Agreement (SLA) violations, especially in FinTech environments where payment failures and security issues need immediate attention. This project proposes the Smart Ticket Understanding Engine, an AI-powered system that automatically classifies each ticket into one of 10 enterprise business categories, analyzes customer sentiment and urgency, assigns an SLA-based priority from P0 to P3, and routes the ticket to the appropriate support queue. Three approaches are evaluated: TF-IDF with Logistic Regression as a baseline, a Bidirectional LSTM (BiLSTM), and a fine-tuned DistilBERT transformer, which is further optimized with ONNX INT8 quantization for low-latency CPU inference. A human-in-the-loop Generative AI component produces contextual draft replies that support agents can review and edit before sending. The system is implemented in Python with FastAPI for backend services, Streamlit for the triage dashboard, SQLite/PostgreSQL for storage, and Docker, MLflow, Evidently AI and Google Cloud Run for deployment, experiment tracking and monitoring. The proposed system is expected to improve routing accuracy, reduce manual triage effort and accelerate responses to critical issues.

3. Introduction

- **Background:** Enterprise and FinTech support teams receive tickets through emails, chat, web forms and other channels. Each ticket is unstructured free text that a support agent must read, understand, categorize, prioritize and route to the right team. Natural Language Processing (NLP) can automate this understanding step.
- **Need for the work:** Manual triage is time-consuming and subjective, so similar tickets are labeled and prioritized differently by different agents. Delays in triage directly increase response time and can breach SLAs.
- **Current challenges:** Ticket text is noisy (typos, mixed tone, missing context), volumes fluctuate, and urgency is often implied rather than stated. In payments and banking, a failed transaction or a security alert must reach the right team within minutes, while a general enquiry can wait.

- **Importance of the proposed system:** The Smart Ticket Understanding Engine combines intent classification, sentiment analysis and SLA-based priority assignment in one pipeline, and adds human-in-the-loop draft replies. It aims to reduce triage effort, improve routing consistency and speed up handling of critical issues while keeping agents in control.

4. Problem Statement

- Support organizations need to convert a continuous stream of unstructured tickets into a category, a sentiment, an SLA priority (P0–P3) and a destination queue accurately and in near real time. Manual triage does this slowly and inconsistently, leading to misrouted tickets, delayed responses and SLA violations.
- **Limitations of existing approaches:** Rule and keyword based routing is brittle and fails on paraphrased or noisy text. Many published systems address only classification and ignore sentiment, urgency and SLA logic [1], [2], [3]. Large transformer models such as BERT give strong accuracy but are costly to serve on CPU-only infrastructure [4], [5].
- **Research gaps:** (i) few end-to-end pipelines that jointly cover intent, sentiment, priority and routing; (ii) limited comparison of classical, recurrent and transformer models on the same ticket data; (iii) limited attention to quantized transformer inference for low-latency CPU deployment [6]; (iv) safe use of generative drafting with human review in support workflows [8].

5. Objectives

- To develop an AI-powered engine that understands support tickets and classifies them into 10 enterprise business categories.
- To analyze customer sentiment and urgency for every ticket.
- To assign an SLA-based priority from P0 (critical) to P3 (low) and route each ticket to the appropriate support queue.
- To compare TF-IDF with Logistic Regression, BiLSTM and fine-tuned DistilBERT using accuracy, precision, recall, macro F1-score and latency.
- To optimize DistilBERT with ONNX INT8 quantization and reduce CPU inference latency and model size with minimal loss of accuracy.
- To provide human-in-the-loop Generative AI draft responses that agents review before sending.
- To deploy the system as containerized microservices with experiment tracking and data-drift monitoring.

6. Literature Review

- Recent and foundational research on ticket automation, intent detection, transformer compression, generative drafting and MLOps was reviewed and compared.
- The comparison below shows that individual components exist, but an integrated, CPU-efficient pipeline with SLA priority and human-in-the-loop drafting is still uncommon.

S. No	Author & Year	Title	Technique Used	Limitations
1	Paramesh & Shreedhara, 2019	Automated IT Service Desk Systems Using Machine Learning Techniques	Machine-learning classification of IT service-desk tickets	Classical text features only; no sentiment, SLA priority or reply assistance
2	Zangari et al., 2023	Ticket Automation: An Insight into Current Research with Applications to Multi-Level Classification Scenarios	Survey of ticket automation and multi-level ticket classification methods	Survey work; no deployable low-latency triage pipeline
3	Casanueva et al., 2020	Efficient Intent Detection with Dual Sentence Encoders	Pretrained dual sentence encoders with light classifiers; introduces the Banking77 dataset	Intent detection only; no sentiment, priority or routing
4	Devlin et al., 2019	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	Deep bidirectional transformer pre-training and task fine-tuning	Large model; high latency and memory cost on CPU
5	Sanh et al., 2019	DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter	Knowledge distillation during pre-training (about 40% smaller, 60% faster)	Small accuracy drop versus BERT; needs further optimization for CPU serving
6	Zafrir et al., 2019	Q8BERT: Quantized 8Bit BERT	8-bit quantization-aware training of BERT	Requires retraining; evaluated on general benchmarks, not support tickets
7	Minaee et al., 2021	Deep Learning–Based Text Classification: A Comprehensive Review	Review of deep text-classification models (CNN, RNN, attention, transformers)	Survey; no domain implementation or deployment guidance
8	Lewis et al., 2020	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Retrieval combined with sequence-to-sequence generation	Needs a knowledge index; risk of unsupported output, so human review is required
9	Zaharia et al., 2018	Accelerating the Machine Learning Lifecycle with MLflow	Experiment tracking, model packaging and registry	Tracking only; no monitoring of text data drift

7. Proposed Methodology

- The system follows an end-to-end NLP pipeline: text preprocessing, tokenization, feature extraction, intent classification, sentiment analysis, priority assignment and automated routing, followed by optional GenAI drafting. Each stage is described below.
- Data Collection
    - Public banking and customer-service intent data (Banking77 [3]) is used as the base and mapped to 10 business categories (for example Payment Failure, Refund/Chargeback, Account Access, Fraud & Security, Card Services, Loan/EMI, KYC/Verification, Billing & Charges, Technical/App Issue, General Enquiry).

    - The data is supplemented with public support-ticket datasets and carefully generated, reviewed synthetic tickets so that every category and priority level is represented.
- Data Preprocessing
    - Lower-casing, removal of HTML, signatures and boilerplate, masking of sensitive tokens (account numbers, phone numbers, e-mail), normalization of whitespace and repeated characters, and stratified train/validation/test splitting.
- Feature Engineering
    - TF-IDF word and character n-grams for the baseline; pretrained word embeddings and padded token sequences for BiLSTM; WordPiece subword tokens for DistilBERT.
    - Extra signals such as sentiment score, urgency keywords, channel and presence of monetary or security terms feed the priority engine.
- Model Design
    - Model 1: TF-IDF with Logistic Regression (baseline). Model 2: BiLSTM classifier. Model 3: fine-tuned DistilBERT [5], exported to ONNX and quantized to INT8 for CPU inference [12].
    - Sentiment and urgency are predicted alongside intent. A priority engine combines category, sentiment, urgency and SLA rules to output P0–P3, and a routing table maps category and priority to a support queue.
    - A GenAI component drafts a contextual reply from the ticket, category and approved response templates; agents must review, edit and approve every draft.
- Training and Testing
    - Models are trained on the training split, tuned on the validation split and evaluated once on a held-out test split. Experiments are logged with MLflow [9].
- Evaluation Metrics
    - Accuracy, precision, recall, weighted and macro F1-score, confusion matrix, priority-assignment accuracy, per-ticket latency (ms), throughput and model size (MB), plus data-drift reports from Evidently AI.

8. System Architecture / Framework

- The architecture diagram below shows the workflow from ticket ingestion to agent review.

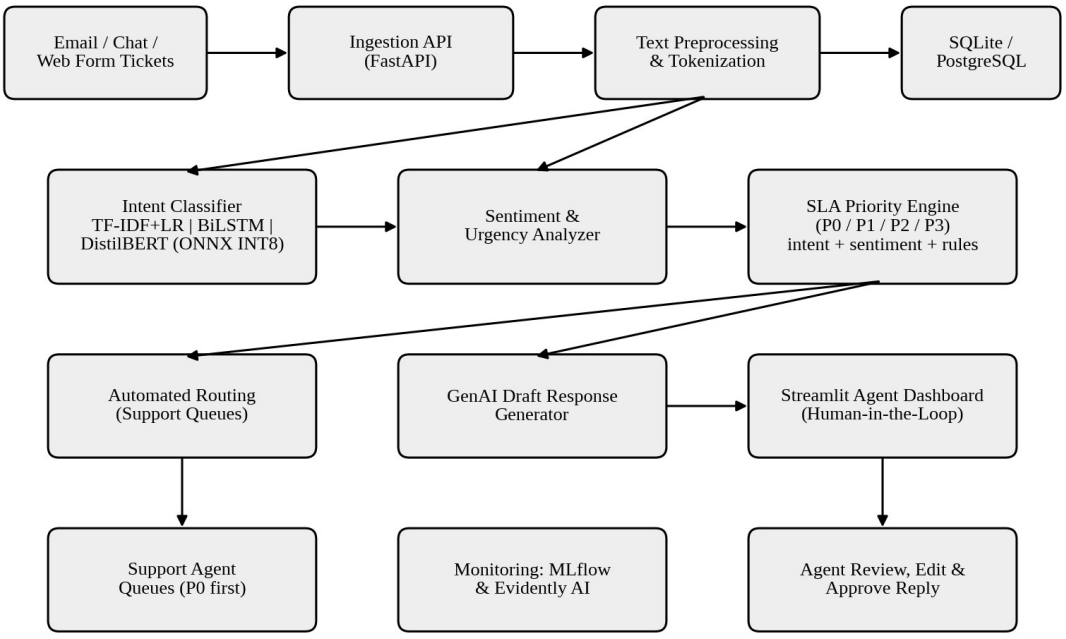


Fig. 1. System architecture of the Smart Ticket Understanding Engine

- **Ingestion Service (FastAPI):** receives tickets from email, chat and web forms through REST endpoints and stores them.
- **Preprocessing Module:** cleans, masks and tokenizes text for the models.
- **Intent Classifier:** predicts one of 10 categories using the selected model (ONNX INT8 DistilBERT in production).
- **Sentiment & Urgency Analyzer:** detects customer tone and urgency cues.
- **SLA Priority Engine and Router:** assigns P0–P3 and places the ticket in the correct support queue.
- **GenAI Draft Generator:** creates a suggested reply for the agent.
- **Streamlit Dashboard:** shows queues sorted by priority, predictions with confidence, and lets agents edit, approve or correct results.
- **MLOps Layer:** MLflow for experiments and model versions, Evidently AI for drift monitoring, Docker and Google Cloud Run for deployment.

9. Algorithms and Technologies Used

Algorithms:

- TF-IDF vectorization with Logistic Regression (baseline)
- Bidirectional LSTM (BiLSTM) for sequence-based text classification
- Fine-tuned DistilBERT transformer [5], [11]
- ONNX INT8 post-training quantization for CPU inference optimization [12]
- Rule-based SLA priority scoring combined with model outputs
- Retrieval/template-grounded generative model for draft replies [8]

Technologies:

- Python
- PyTorch, Hugging Face Transformers [10], ONNX Runtime

- Scikit-learn [13], NLTK/spaCy for preprocessing
- FastAPI (backend microservices) and Streamlit (dashboard)
- SQLite / PostgreSQL
- Docker, Google Cloud Run, MLflow [9], Evidently AI, Git/GitHub

10. Dataset Description

- **Source:** Banking77 [3] (13,083 customer banking queries labeled with 77 intents) as the base, mapped to 10 business categories; supplemented with public customer-support ticket data and reviewed synthetic tickets.
- **Number of samples:** approximately 15,000–20,000 labeled tickets are planned after merging and balancing; the exact count will be reported after data preparation.
- **Attributes / features:** ticket ID, channel, subject, body text, category (10 classes), sentiment (negative / neutral / positive), priority (P0–P3), target queue and timestamp.
- **Labeling:** categories come from the mapped intents; sentiment is pre-labeled with a pretrained model and verified on a manual sample; priorities follow documented SLA rules and are validated by team review.
- **Training / testing split:** 70% training, 15% validation and 15% testing, stratified by category.

11. Experimental Setup

- **Hardware:** training on a cloud GPU (Google Colab or Kaggle, NVIDIA T4 class); latency benchmarks on a standard CPU-only machine to reflect deployment.
- **Software:** Python 3.10+, PyTorch, Hugging Face Transformers, ONNX Runtime, scikit-learn, FastAPI, Streamlit, Docker, MLflow and Evidently AI.

Hyperparameters (initial values, to be tuned on the validation set):

- TF-IDF + Logistic Regression: word n-grams (1,2), up to 50,000 features, regularization C tuned over {0.5, 1, 5, 10}.
- BiLSTM: embedding size 300, hidden size 128, dropout 0.3, Adam optimizer, learning rate 1e-3, 10–15 epochs.
- DistilBERT: max sequence length 128, batch size 32, AdamW with learning rate 2e-5, weight decay 0.01, 10% warm-up, 3–5 epochs.
- **Performance metrics:** accuracy, precision, recall, macro F1-score, confusion matrix, latency, throughput and model size.

12. Expected Results

The values below are targets to be verified experimentally; final figures will be reported after testing.

- Accuracy: DistilBERT expected to outperform the TF-IDF and BiLSTM models, with a target of about 90% or higher on the 10-class task.
- Precision and Recall: balanced values across categories, with particularly high recall for P0/P1 tickets so critical issues are not missed.
- F1-Score: macro F1-score close to accuracy, showing consistent performance on smaller categories.

- Reduced computational complexity: ONNX INT8 quantization expected to cut model size and CPU latency substantially (roughly 2–4x) with an accuracy loss of about 1% or less.
- Improved scalability: stateless FastAPI services in Docker containers on Cloud Run for real-time, high-throughput triage.
- Reduced manual triage effort through automated routing and agent-approved draft replies.

13. Conclusion

The Smart Ticket Understanding Engine is proposed as an integrated, practical solution for enterprise and FinTech ticket management. It combines intent classification, sentiment analysis, SLA-based P0–P3 prioritization and automated routing, and evaluates classical, recurrent and transformer models on the same data. Quantizing DistilBERT with ONNX INT8 makes accurate transformer inference feasible on low-cost CPU infrastructure, and human-in-the-loop draft replies reduce repetitive agent work while keeping human oversight. The project is expected to improve routing accuracy, shorten response times for critical issues and provide a scalable, monitored deployment.

14. Timeline / Work Plan

Phase	Activity	Duration
Phase 1	Literature Survey	2 Weeks
Phase 2	Data Collection	2 Weeks
Phase 3	Model Development	4 Weeks
Phase 4	Testing & Evaluation	3 Weeks
Phase 5	Documentation	2 Weeks

Note : all the phases will cover in Two Reviews

15. References

[1] S. P. Paramesh and K. S. Shreedhara, “Automated IT service desk systems using machine learning techniques,” in *Data Analytics and Learning*, Lecture Notes in Networks and Systems, vol. 43. Springer, 2019, pp. 331–346.

[2] A. Zangari, M. Marcuzzo, M. Schiavinato, A. Gasparetto, and A. Albarelli, “Ticket automation: An insight into current research with applications to multi-level classification scenarios,” *Expert Systems with Applications*, vol. 225, Art. no. 119984, 2023.

[3] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, “Efficient intent detection with dual sentence encoders,” in *Proc. 2nd Workshop on NLP for Conversational AI (NLP4ConvAI)*, ACL, 2020, pp. 38–45.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” *arXiv:1910.01108*, 2019.

[6] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, “Q8BERT: Quantized 8Bit BERT,” in *Proc. 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC2-NIPS)*, 2019.

[7] S. Minaee *et al.*, “Deep learning–based text classification: A comprehensive review,” *ACM Computing Surveys*, vol. 54, no. 3, Art. 62, 2021.

[8] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[9] M. Zaharia *et al.*, “Accelerating the machine learning lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[10] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[11] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[12] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.

[13] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.